

ServiceNow GlideForm (g_form) Reference Manual

Detailed Guide with Why/Where Explanations and Code Examples

This reference manual provides an in-depth breakdown of the GlideForm (g_form) API methods. For each method, you will find its functional utility, its context inside client workflows, and production-ready code examples.

1. Value Manipulation APIs

A. g_form.getValue(fieldName)

Reads the current input value of a field dynamically inside browser session memory.

WHY USED: To inspect and check the user's data values before triggering conditional fields or enforcing record validations.

WHERE USED: In onChange and onSubmit client scripts to run custom form evaluations.

```
var state = g_form.getValue('state');
if (state === '3') { // 3 = On Hold
    jslog('Ticket is currently on hold.');
```

B. g_form.setValue(fieldName, value, displayValue)

Sets the value of a target field. For Reference fields, pass the display value as the third parameter to prevent additional database read queries.

WHY USED: To auto-populate assignments, categories, or descriptions based on other fields or selections.

WHERE USED: In onLoad or onChange scripts to save manual typing for form users.

```
g_form.setValue('short_description', 'Auto-generated title');
// Reference field optimization:
g_form.setValue('assigned_to', '5137153ec611227c0000851e3e7f2235', 'John Doe');
```

C. g_form.clearValue(fieldName)

Clears the value of a target field.

WHY USED: To wipe out stale or invalid selections when fields are reset or parent dropdown values change.

WHERE USED: In onChange scripts where parent-child field logic is defined.

```
if (g_form.getValue('category') === '') {
    g_form.clearValue('subcategory');
```

D. g_form.getUniqueValue()

Returns the unique sys_id string of the record currently displayed on the form.

WHY USED: To retrieve the record's primary key without needing to inspect a specific form field.

WHERE USED: In client scripts or custom client-side UI Actions to trigger GlideAjax queries.

```
var recordID = g_form.getUniqueValue();
jslog('Current Record Sys ID: ' + recordID);
```

E. g_form.isNewRecord()

Returns true if the record is brand new and has not yet been saved to the database.

WHY USED: To execute initialization scripts or set default values only for new creations, avoiding overwrites on saved tickets.

WHERE USED: In onLoad Client Scripts.

```
if (g_form.isNewRecord()) {
    g_form.setValue('description', 'Default templates applied.');
```

2. Visibility & Display APIs

A. g_form.setDisplay(fieldName, true/false)

Hides or displays a field, dynamically collapsing or expanding form layouts (Best Practice).

WHY USED: To create clean, dynamic forms without showing fields that are not applicable to the current state.

WHERE USED: In onChange or onLoad scripts when dealing with conditional forms.

```
g_form.setDisplay('hold_reason', false); // Hides and collapses space
```

B. g_form.setVisible(fieldName, true/false)

Hides or displays a field, but leaves the blank space where the field resided intact.

WHY USED: To keep column grids aligned in multi-column layouts when hiding specific fields.

WHERE USED: In specialized onLoad forms where spacing consistency is critical.

```
g_form.setVisible('u_secondary_contact', false); // Leaves empty space
```

C. g_form.setSectionDisplay(sectionName, true/false)

Hides or displays entire form sections/tabs dynamically.

WHY USED: To show or hide entire tabs of fields (e.g., Resolution Information) depending on the ticket state or category.

WHERE USED: In onLoad or onChange scripts where major groups of fields are conditional.

```
var state = g_form.getValue('state');
if (state !== '6' && state !== '7') {
    g_form.setSectionDisplay('resolution_information', false);
}
```

D. g_form.isSectionVisible(sectionName)

Returns true if the specified form section or tab is currently visible.

WHY USED: To condition script actions on whether a tab is displayed to the user.

WHERE USED: In onLoad or onChange verification routines.

```
if (g_form.isSectionVisible('resolution_information')) {
    g_form.setMandatory('close_notes', true);
}
```

3. Mandatory, Read-Only & Disabled Controls

A. g_form.setMandatory(fieldName, true/false)

Enforces database-level field requirement flags prior to submission.

WHY USED: To force users to supply descriptions, close codes, or justification details before submitting.

WHERE USED: In onChange or onSubmit scripts to maintain data quality thresholds.

```
g_form.setMandatory('close_notes', true);
```

B. g_form.setReadOnly(fieldName, true/false)

Locks or unlocks form fields dynamically.

WHY USED: To prevent manual edits on system-populated parameters, reference fields, or archived states.

WHERE USED: On saved records or fields modified only by scripts and integrations.

```
if (!g_form.isNewRecord()) {
    g_form.setReadOnly('caller_id', true);
}
```

C. g_form.setDisabled(fieldName, true/false)

Older legacy alternative to setReadOnly() that fully locks the UI element.

WHY USED: To completely gray out fields, signaling total lock; however, setReadOnly() is preferred for user accessibility copy-paste compatibility.

WHERE USED: In client scripts where fields must be styled as inactive.

```
g_form.setDisabled('u_alternate_email', true);
```

4. Dynamic Field Labels & Messaging

A. g_form.setLabel(fieldName, labelString)

Modifies the visible text label of a field dynamically.

WHY USED: To update terminology depending on the task category or record context.

WHERE USED: In onLoad or onChange scripts on generic CI or description fields.

```
if (g_form.getValue('category') === 'software') {
    g_form.setLabel('cmdb_ci', 'Target Software Application');
}
```

B. g_form.getLabelOf(fieldName)

Returns the current display label of the specified field.

WHY USED: To construct user-friendly validation messages referencing field labels dynamically.

WHERE USED: In onSubmit scripts for clear error message generation.

```
var descLabel = g_form.getLabelOf('description');
g_form.addErrorMessage('Please fill in required field: ' + descLabel);
```

C. g_form.showFieldMsg(fieldName, message, type)

Displays styled validation feedback inline below a target field.

WHY USED: To guide users with small warnings, input validations, or tips without block popups.

WHERE USED: To display field-level warning constraints (e.g. invalid phone number).

```
g_form.showFieldMsg('u_email', 'Please double-check email domain', 'warning');
```

D. g_form.hideFieldMsg(fieldName)

Removes currently active inline field messages.

WHY USED: To clear warnings once correct values are typed into inputs.

WHERE USED: In onChange validation routines.

```
g_form.hideFieldMsg('u_email');
```

E. Form-Wide Banners

g_form.addInfoMessage(), g_form.addErrorMessage(), g_form.clearMessages()

Controls global success/error banners displayed at the top of the form.

WHY USED: To alert users of blocking errors, save confirmations, or status updates on submit.

WHERE USED: Inside onLoad or onSubmit scripts to broadcast global state validations.

```
g_form.clearMessages();
g_form.addErrorMessage('Error: Selected schedule conflicts with active maintenance.');
```

5. Asynchronous Reference Field Records Access

A. g_form.getReference(fieldName, callback)

Queries details on tables referenced by form fields asynchronously.

WHY USED: To pull customer fields (VIP status, department, manager) without blocking the UI main thread.

WHERE USED: Inside onChange Client Scripts on Caller or Configuration Item changes.

Chaining Warning (getReference().property):

Chaining directly like g_form.getReference('caller_id').email runs a SYNCHRONOUS database lookup that freezes the browser. Always retrieve fields asynchronously in the callback function.

```
g_form.getReference('caller_id', function(caller) {
    var emailAddress = caller.email; // Safe Asynchronous Email Access
    if (emailAddress) {
        g_form.setValue('u_alternate_email', emailAddress);
    }
});
```

6. Choice List Manipulation

A. g_form.clearOptions(fieldName)

Removes all options from a dropdown choice field.

WHY USED: To wipe out old, stale, or conflicting choice selections prior to building dynamic sub-categories.

WHERE USED: In onChange scripts where parent fields dictate valid dropdown categories.

```
g_form.clearOptions('subcategory');
```

B. g_form.addOption(fieldName, choiceValue, choiceLabel, choiceIndex)

Dynamically appends a choice option to a dropdown list at the specified 0-based index.

WHY USED: To populate select boxes dynamically depending on user attributes, roles, or other fields.

WHERE USED: In onChange dropdown management scripts.

```
g_form.addOption('subcategory', 'email_issue', 'Email & Collaboration', 0);
g_form.addOption('subcategory', 'network_issue', 'VPN & Networks', 1);
```

C. g_form.removeOption(fieldName, choiceValue)

Deletes a specific choice option from a dropdown list.

WHY USED: To prevent specific selections based on user status or record context dynamically.

WHERE USED: To hide Closed/Resolved states for non-admins.

```
if (!g_form.hasRoleExactly('admin')) {
    g_form.removeOption('state', '7'); // 7 = Closed
}
```

7. Utility APIs & Form Actions

A. Utility Definitions

getFieldNames(), isMandatory(), isVisible(), isReadOnly()

Methods that audit layout field names and inspect active mandatory/visibility properties.

WHY USED: To bulk lock layout properties or check settings before modifying values.

WHERE USED: Conditional forms with complex toggle workflows.

```
var fields = g_form.getFieldNames();
for (var i = 0; i < fields.length; i++) {
    if (!g_form.isReadOnly(fields[i])) {
        g_form.setReadOnly(fields[i], true);
    }
}
```

B. g_form.save() & g_form.submit()

save() writes edits to the database and keeps the form active. submit() writes edits and redirects to list view.

WHY USED: To save values programmatically inside custom UI buttons or form validations.

WHERE USED: Client-side UI Actions and custom form actions.

```
g_form.setValue('short_description', 'Updated via script click');
g_form.save();
```